ACHELION CAPITAL MANAGEMENT, LLC

ARMS Module Specification

Two New Modules — Addendum to FSD v1.1

Module A: PTRH Full Automation (Tier 0)
Module B: Defensive Sleeve Harvest Protocol (DSHP)

For: Josh Paul — Lead Developer
Version: 1.0 — March 26, 2026
Classification: CONFIDENTIAL — GP and Development Use Only

These two modules close two identified gaps in the ARMS v4.0 architecture. Both are fully buildable today with zero new dependencies. Build Module A first — it is simpler and eliminates a recurring daily decision queue item immediately.

"Silence is trust in the architecture."

0. Why These Two Modules Exist

0.1 The PTRH Gap

The Permanent Tail Risk Hedge (PTRH) has fully deterministic rules in PM Playbook v4.0: sizing multipliers are defined by regime, the roll trigger is defined at 30 DTE, and regime-transition resizing is automatic. Despite this, PTRH sizing confirmation has appeared in the PM decision queue every morning because the connection between the rules and the execution pipeline does not yet exist.

A decision queue item that requires no human judgment is not a safeguard. It is latency. The PM reviewing a sizing confirmation that the system already knows the answer to is the precise failure mode ARMS is designed to eliminate. Module A closes this gap permanently. PTRH becomes fully Tier 0. It disappears from the decision queue. The PM sees its status in the module panel as confirmation it ran correctly — not as a request for approval.

One legitimate exception is preserved: a one-time coverage review when a dual-risk scenario is identified — such as the current Iran war plus Warsh nomination combination — that may require a coverage standard update. This is Tier 2: PM confirms the updated standard once, and the system enforces it going forward. It is not a recurring daily item.

0.2 The Defensive Sleeve Harvest Gap

The defensive sleeve is designed for capital preservation, not return generation. SGOV holds value. SGOL hedges inflation and geopolitical risk. DBMF captures trend momentum. STRC generates yield. None of these instruments are in the portfolio to produce alpha. That is their design.

But markets do not respect instrument intent. Under specific conditions — war bids, commodity super-cycles, yield compression, trend momentum — defensive instruments generate significant price appreciation well beyond their designed function. When this happens, the current ARMS architecture has no mechanism to act. PERM only applies to the equity book. The defensive sleeve sits outside it.

The consequence is predictable: an instrument that was bought as insurance becomes a speculative position through appreciation, its weight drifts above its target, its defensive function is diluted, and when the conditions that drove the gain reverse — as they always do — the gain evaporates without having been captured. The architecture watched the gain appear and watched it disappear.

Module B closes this gap. The Defensive Sleeve Harvest Protocol extends gain-harvesting discipline into the defensive sleeve with instrument-appropriate thresholds and a simple rebalancing mechanic. It does not change the sleeve's defensive character. It ensures that when the sleeve produces unexpected gains, those gains are crystallized rather than surrendered.

MODULE A — PTRH FULL AUTOMATION

Permanent Tail Risk Hedge — Tier 0 Autonomous Operation

1. Module A Overview

File	src/engine/tail_hedge.py — update existing module
Tier classification	Tier 0 — fully autonomous. No PM veto window. No decision queue entry.
Build time	2 days — all rules already defined in PM Playbook v4.0. This is connection work, not design work.
Dependencies	ARAS regime output · Brokerage API (IBKR) · market data feed for QQQ options chain
Ongoing cost	$0 — no new APIs or feeds required
Decision queue impact	PTRH sizing confirmation removed from daily queue entirely after deployment
Exception	One-time dual-risk coverage review (Tier 2) — see Section 1.4

1.1 The Four Autonomous PTRH Actions

Every PTRH action is deterministic. The inputs are known. The outputs are defined. There is no ambiguity that requires human judgment. All four actions are Tier 0 from deployment.

Action	Trigger condition	Rule	Execution	Audit
Regime sizing	Regime transition confirmed by ARAS	Apply multiplier table (Section 1.2)	Same session — IBKR order submitted immediately	Log: regime_from, regime_to, old_size, new_size, timestamp
DTE roll	Any QQQ put position reaches 30 DTE	Buy next expiry at same strike and same notional. Sell expiring position.	Same session as trigger	Log: old_expiry, new_expiry, strike, notional, cost_of_roll
Coverage verification	Daily at 5:25 AM CT as part	Confirm notional coverage	Advisory only if within tolerance.	Log: expected_notional, actual_notional, drift_pct

	of module sweep	matches regime-required multiplier. Flag if drift >5%.	Auto-correct if >5% drift.	
Strike selection	New position or roll requires strike selection	ATM strike at time of execution. No discretion. No optimization attempt.	Automatic at market execution	Log: QQQ_price_at_execution, strike_selected

1.2 Regime Sizing Multiplier Table

This table is the complete rule set. No other inputs are required. The system reads the current regime from ARAS and sizes accordingly.

Regime	Score range	PTRH multiplier	NAV % in puts	Rationale
RISK_ON	≤0.30	**1.0×**	**1.2%**	Minimum permanent insurance. Always on. Baseline cost of being in the market.
WATCH	0.31–0.50	**1.0×**	**1.2%**	Same as RISK_ON. Conditions are positive but watched. Insurance cost unchanged.
NEUTRAL	0.51–0.65	**1.25×**	**1.5%**	Environment mixed. Begin adding insurance before conditions worsen further.
DEFENSIVE	0.66–0.80	**1.5×**	**1.8%**	Active protection regime. Meaningful downside risk present. Insurance material.
CRASH	>0.80	**2.0×**	**2.4%**	Maximum insurance deployment. Portfolio under acute stress. Every basis point of protection matters.

Base notional = 1.2% of current NAV in QQQ ATM puts. Each multiplier is applied to this base. Coverage verification runs daily at 5:25 AM CT and auto-corrects if actual notional drifts more than 5% from the regime-required level due to NAV changes.

1.3 Dual-Risk Coverage Standard (One-Time Tier 2 Review)

When a dual-risk scenario is present — defined as two simultaneous independent downside pressures on QQQ, such as the current Iran war plus Kevin Warsh hawkish rate overlay — the PM executes a

one-time coverage standard review. This is not a recurring daily item. It occurs once per identified dual-risk event.

The review determines whether the current PTRH notional provides adequate coverage for the dual-risk scenario. The standard question: does the existing put notional provide meaningful payoff if QQQ falls to the dual-risk downside estimate? If the current standard covers −15% and the dual-risk scenario implies −20% to −25%, the PM sets a new coverage standard. The system then enforces that standard automatically going forward until the dual-risk event resolves.

Trigger	Two simultaneous independent downside pressures identified by the PM or by ARMS signal analysis
Process	PM reviews current notional coverage vs dual-risk downside estimate. Documents updated standard in session log.
System action	ARMS adjusts PTRH notional to meet the updated standard. Holds until PM signals dual-risk resolved.
Resolution	When dual-risk event resolves — e.g., Iran ceasefire confirmed AND Warsh confirmation complete — system reverts to regime-standard multiplier automatically.
Tier classification	Tier 2 — PM confirms once. System executes and maintains.

1.4 What Disappears From the Decision Queue

After Module A is deployed, PTRH never appears in the PM decision queue under normal conditions.

The daily monitor module status panel shows PTRH status as a confirmation: current multiplier, current notional, DTE of nearest expiry, last roll date. The PM reads this as a status report, not an action item. If Module A encounters an execution failure — broker API error, insufficient liquidity, options market closed — an ALERT fires in the module status panel and a DANGER alert appears in the alerts section. The PM acts on failures, not on normal operations.

1.5 Implementation Specification

Data structures

```
@dataclass class PTRHStatus:
    regime: str                    # current canonical regime label
    multiplier: float              # current sizing multiplier
    target_notional_pct: float     # % of NAV in puts
    actual_notional_pct: float     # actual current coverage
    coverage_drift_pct: float      # abs(target - actual)
    nearest_expiry_dte: int        # days to expiration of nearest position
    last_roll_date: str            # ISO date of most recent roll
    dual_risk_active: bool         # PM-declared dual-risk flag
    dual_risk_standard_pct: float  # override notional if dual_risk_active
    last_action: str               # 'NONE'|'ROLL'|'RESIZE'|'CORRECT_DRIFT'
    last_action_timestamp: str
```

Core logic

```
def run_ptrh_module(regime: str, nav: float,
                    current_positions: List[OptionsPosition],
                    dual_risk_override: float = None) -> PTRHStatus:

    # 1. Determine target notional
    base_pct   = MULTIPLIER_TABLE[regime] * BASE_NOTIONAL_PCT
    target_pct = dual_risk_override if dual_risk_override else base_pct
    target_notional = nav * target_pct

    # 2. Check DTE — roll if any position at or below 30 days
    for pos in current_positions:
        if pos.dte <= 30:
            execute_roll(pos, target_notional)
            log_action('ROLL', pos, target_notional)

    # 3. Check notional drift — correct if > 5%
    actual_pct   = sum(p.notional for p in current_positions) / nav
    drift        = abs(target_pct - actual_pct)
    if drift > 0.05:
        execute_notional_correction(target_notional, current_positions)
        log_action('CORRECT_DRIFT', drift=drift)

    # 4. Return status for monitor display
    return PTRHStatus(regime=regime, multiplier=MULTIPLIER_TABLE[regime],
        target_notional_pct=target_pct, actual_notional_pct=actual_pct,
        coverage_drift_pct=drift, ...)
```

Scheduler entry

```
# runs inside daily module sweep at 5:25 AM CT
```

```
@flow(name='ptrh_daily_check')
def ptrh_daily_flow():
    regime  = get_current_regime()
    nav     = get_portfolio_nav()
    pos     = broker_api.get_qqq_put_positions()
    dual    = config.get('dual_risk_override_pct', None)
    status  = run_ptrh_module(regime, nav, pos, dual)
    db.save_ptrh_status(status)


# intraday regime transition trigger — fires immediately on ARAS output
@flow(name='ptrh_regime_transition')
def ptrh_regime_transition_flow(old_regime: str, new_regime: str):
    if MULTIPLIER_TABLE[new_regime] != MULTIPLIER_TABLE[old_regime]:
        nav = get_portfolio_nav()
        pos = broker_api.get_qqq_put_positions()
        run_ptrh_module(new_regime, nav, pos)
```

MODULE B — DEFENSIVE SLEEVE HARVEST PROTOCOL

DSHP — Gain Crystallization in the Defensive Sleeve

2. Module B Overview

File	src/engine/dshp.py — new module. Extends perm.py logic into defensive sleeve.
Tier classification	Tier 1 — default execute with PM veto window. System queues trim. PM has 4-hour window. Silence = execute.
Build time	1-2 days — extends existing PERM module logic. Same gain-measurement pattern, different thresholds and mechanic.
Dependencies	Portfolio snapshot (NAV + sleeve positions) · market data feed for sleeve instrument prices · brokerage API for execution
Ongoing cost	$0 — no new dependencies
PERM relationship	DSHP is a companion to PERM, not a replacement. PERM governs equity book positions (covered calls on >30% gains). DSHP governs defensive sleeve positions (trim-to-target on >20% appreciation). Different instruments, different mechanics, same philosophy: gains are crystallized systematically.

2.1 The Core Principle

When a defensive instrument appreciates beyond its designed function, it has become a speculative position. DSHP converts it back into a defensive position and banks the gain.

A 2% allocation to SGOL that appreciates to 2.8% is no longer functioning as a 2% defensive hedge. It is a 2.8% allocation to gold — and the excess 0.8% is now exposed to the same reversal risk that produced the gain. Left unmanaged, this is a guaranteed outcome: the gain appeared, the gain will reverse, the architecture did nothing. DSHP prevents this.

The mechanic is simple: when a sleeve instrument appreciates beyond its harvest threshold, DSHP trims it back to its target weight and redeposits the proceeds into SGOV or cash within the defensive sleeve. The total sleeve allocation remains 14%. The instrument returns to its designed weight and function. The gain is in SGOV, not at risk.

2.2 Instrument-Specific Rules

Each instrument in the defensive sleeve has its own harvest threshold, mechanic, and rationale. They are not treated identically because they are not identical instruments.

Instrument	Target wt.	Harvest threshold	Mechanic	Proceeds to	Rationale
SGOL	2.0%	>20% appreciation above entry value of the position	Trim to 2.0% target weight. Partial sell — not full exit.	SGOV (within defensive sleeve)	Gold appreciation is driven by war bids and inflation fears — both mean-revert. Crystallize the gain. Maintain the hedge at its designed weight.
DBMF	5.0%	>15% appreciation above entry OR weight drift >1.5pp above 5.0% target	Trim to 5.0% target weight. Partial sell.	SGOV (within defensive sleeve)	Managed futures gains are trend-captured — they reverse when trends reverse. The dual trigger (appreciation OR weight drift) catches both slow appreciation and sharp moves. Oil trend reversal risk makes this timely.
STRC	4.0% + reserve	No price harvest trigger. Yield only.	No trim action. Yield income distributed to cash sleeve per existing rules.	N/A	STRC is a yield instrument, not a price appreciation instrument. Its value is the income stream at 11.5% yield with NAV intact. You do not harvest a bond yield. You let it pay.
SGOV	3.0%	No harvest trigger. Receives DSHP proceeds.	Acts as the proceeds destination for SGOL and DBMF harvests.	N/A — receives	T-bill ETF. Near-zero volatility. Harvested gains are safe here. Sleeve total remains 14%. No architecture change required.

2.3 Harvest Trigger Logic

DSHP runs daily at 5:25 AM CT as part of the module sweep. It checks each eligible instrument against its trigger condition. When a trigger fires, DSHP queues a Tier 1 trim action with a 4-hour veto window.

```
def check_dshp_triggers(sleeve_positions: Dict,
                        nav: float) -> List[HarvestAction]:
    actions = []

    # SGOL — appreciation trigger
    sgol_pos = sleeve_positions['SGOL']
    sgol_appreciation = (sgol_pos.current_value - sgol_pos.entry_value)
```

```
                    / sgol_pos.entry_value
if sgol_appreciation > SGOL_HARVEST_THRESHOLD:  # 0.20
    target_value  = nav * SGOL_TARGET_WEIGHT    # 0.02
    trim_amount   = sgol_pos.current_value - target_value
    actions.append(HarvestAction(
        instrument='SGOL',
        action_type='TRIM_TO_TARGET',
        trim_amount=trim_amount,
        proceeds_to='SGOV',
        trigger='APPRECIATION',
        appreciation_pct=sgol_appreciation,
        tier=1,
        veto_window_hours=4.0
    ))

# DBMF — appreciation OR weight drift trigger
dbmf_pos = sleeve_positions['DBMF']
dbmf_appreciation = (dbmf_pos.current_value - dbmf_pos.entry_value)
                    / dbmf_pos.entry_value
dbmf_weight = dbmf_pos.current_value / nav
dbmf_drift  = dbmf_weight - DBMF_TARGET_WEIGHT  # 0.05

if dbmf_appreciation > DBMF_HARVEST_THRESHOLD   # 0.15
or dbmf_drift > DBMF_DRIFT_THRESHOLD:           # 0.015
    target_value  = nav * DBMF_TARGET_WEIGHT
    trim_amount   = dbmf_pos.current_value - target_value
    trigger_type  = 'APPRECIATION' if dbmf_appreciation > 0.15
                    else 'WEIGHT_DRIFT'
    actions.append(HarvestAction(
        instrument='DBMF',
        action_type='TRIM_TO_TARGET',
        trim_amount=trim_amount,
        proceeds_to='SGOV',
        trigger=trigger_type,
        tier=1,
        veto_window_hours=4.0
    ))

return actions
```

2.4 Tier 1 Confirmation Queue Integration

Every DSHP harvest action routes through the existing Tier 1 confirmation queue (src/execution/confirmation_queue.py). The three-option interface applies identically to DSHP as to any Tier 1 action:

- Option 1 — No new information, execute: PM has no reason to block. Trim executes immediately. Session log records 'PM confirmed — no new information.'

- Option 2 — New information present, hold: PM has a specific reason to believe the harvest should not occur at this time. PM documents the reason. Action is paused for up to 24 hours for PM to provide updated standard.
- Option 3 — Veto with rationale: PM blocks the harvest permanently for this trigger event. Written rationale required. If PM vetoes more than 2 DSHP actions in a 30-day period, a GP review is triggered — this is a signal that either the thresholds need recalibration or the PM is accumulating speculative exposure in the defensive sleeve.
- No response within 4 hours: harvest executes automatically. Silence is trust in the architecture.

2.5 Harvest Action Data Structure

```
@dataclass class HarvestAction:
    instrument: str            # 'SGOL' | 'DBMF'
    action_type: str           # 'TRIM_TO_TARGET'
    trim_amount: float         # dollar amount to sell
    proceeds_to: str           # 'SGOV' — always
    trigger: str               # 'APPRECIATION' | 'WEIGHT_DRIFT'
    appreciation_pct: float    # gain % at time of trigger
    weight_at_trigger: float   # actual weight when triggered
    target_weight: float       # instrument target weight
    nav_at_trigger: float      # portfolio NAV at trigger
    tier: int                  # always 1
    veto_window_hours: float   # always 4.0
    queued_at: str             # ISO timestamp
    executed_at: str           # None until execution
    pm_response: str           # 'EXECUTE'|'HOLD'|'VETO'|'TIMEOUT'
    pm_rationale: str          # required if HOLD or VETO
```

2.6 Configuration Constants

All thresholds are stored in a configuration file and can be adjusted by the PM without code changes. Josh should implement a config validation step that alerts if any threshold is set outside a reasonable range.

```
# src/config/dshp_config.py
SGOL_TARGET_WEIGHT      = 0.020   # 2.0% of NAV
SGOL_HARVEST_THRESHOLD  = 0.200   # harvest if >20% appreciation

DBMF_TARGET_WEIGHT      = 0.050   # 5.0% of NAV
DBMF_HARVEST_THRESHOLD  = 0.150   # harvest if >15% appreciation
DBMF_DRIFT_THRESHOLD    = 0.015   # harvest if weight drifts >1.5pp above target

# STRC and SGOV: no harvest triggers — yield and receiver respectively

DSHP_VETO_WINDOW_HOURS  = 4.0     # Tier 1 veto window
DSHP_VETO_GP_ALERT_THRESHOLD = 2  # GP review if >2 vetoes in 30 days
```

2.7 Daily Monitor Integration

DSHP status appears in the defensive sleeve section of the daily monitor with the following additions:

Field	Content	Source	Render rule
SGOL gain	Current unrealized appreciation % above entry	DSHP module output	Green if <15% · Amber 15-19% · Red if >20% (harvest imminent)
DBMF gain	Current unrealized appreciation % + weight drift	DSHP module output	Green if clear · Amber if approaching either threshold
DSHP status	Last harvest date + amount + instrument	DSHP session log	'Last harvest: [INSTRUMENT] [DATE] [AMOUNT]' or 'No harvest triggered'
Pending harvest	If Tier 1 action is queued and veto window is open	Confirmation queue	AMBER alert in alerts section with veto window countdown

3. Combined Build Plan for Josh

Build Module A first. It is simpler, eliminates an immediate daily friction, and validates the broker API execution loop before the more complex DSHP logic is added.

Module	Build time	Priority	Files	Milestone
Module A PTRH Automation	2 days	Build first	src/engine/tail_hedge.py (update existing)	PTRH removed from decision queue. Tier 0 confirmed working. Roll and resize executing automatically.
Module B DSHP	1-2 days	Build second	src/engine/dshp.py (new file) src/config/dshp_config.py (new file)	First DSHP harvest queued and executed through Tier 1 confirmation queue. Defensive sleeve gains crystallized systematically.

3.1 Testing Protocol — Module A

1. Force a regime transition in paper mode: RISK_ON → DEFENSIVE. Verify PTRH multiplier changes from 1.0× to 1.5× automatically. Verify broker API submits the correct order. Verify session log captures the transition with old and new sizing.
2. Set a test position to 31 DTE. Verify the roll fires automatically. Verify the new position is at the same strike and notional. Verify session log captures old expiry, new expiry, and cost of roll.
3. Manually drift the notional to >5% below target in paper mode. Verify the auto-correction fires within the next daily sweep. Verify session log captures drift_pct and correction action.
4. Simulate a broker API failure. Verify that a DANGER alert fires in the module status panel and the alerts section. Verify that the daily monitor shows PTRH status as ALERT, not as a decision queue item.

3.2 Testing Protocol — Module B

5. Set SGOL appreciation to 21% above entry in test data. Verify DSHP fires a Tier 1 harvest action. Verify the confirmation queue receives it with a 4-hour veto window. Verify the trim amount equals the difference between current value and target weight value.
6. Let the veto window expire with no PM response. Verify the trim executes automatically. Verify proceeds are allocated to SGOV. Verify total sleeve allocation remains 14%.
7. Set DBMF weight to 6.6% (1.6pp above 5.0% target — exceeds 1.5pp drift threshold). Verify DSHP fires on weight drift trigger, not appreciation trigger. Verify session log records trigger type as WEIGHT_DRIFT.

8. Submit a PM veto via Option 3. Verify the harvest is blocked. Verify written rationale is required. Verify the action is logged as VETO with rationale captured.
9. Submit 3 vetoes in a 30-day window in test mode. Verify GP review alert fires automatically on the third veto.

3.3 File Structure Update

The following files are added or updated by these two modules. Update the FSD v1.1 file structure table accordingly.

File	Status	Change
src/engine/tail_hedge.py	UPDATED	Full Tier 0 automation added. Regime sizing table, DTE roll logic, drift correction, dual-risk override support.
src/engine/dshp.py	NEW	Defensive Sleeve Harvest Protocol. SGOL and DBMF triggers, Tier 1 queue integration, SGOV reallocation mechanic.
src/config/dshp_config.py	NEW	All DSHP threshold constants. Adjustable without code changes.
src/reporting/daily_monitor.py	UPDATED	DSHP gain tracking fields added to defensive sleeve section. PTRH status display updated to confirmation panel (not decision queue).
src/execution/confirmation_queue.py	UPDATED	DSHP harvest actions added as a new Tier 1 action type. No structural changes required — same queue mechanic.

After these two modules are deployed: PTRH runs itself. The defensive sleeve harvests its own gains. The daily monitor reflects both as confirmations, not requests. The blade is sharper.

"Silence is trust in the architecture."

Achelion Capital Management, LLC · Flow. Illumination. Discipline. Conviction.